A ideia principal do programa é simular milhares de rodadas de um jogo fictício, utilizando probabilidades para calcular como um saldo virtual evolui ao longo do tempo.

1. O que o programa faz?

O simulador permite configurar quatro parâmetros:

Parâmetro	Função
PRIMEs por rodada	Quantidade ganha ou perdida em cada rodada
Número de rodadas	Quantas vezes o jogo será simulado
Chance de vitória	Probabilidade de ganhar cada rodada
Saldo inicial	Quantidade inicial de PRIMEs virtuais

Por exemplo, imagine esta configuração:

- Aposta por rodada: 100 PRIMEs.
- Número de rodadas: 10.000.
- Chance de vitória: 49%.
- Saldo inicial: 1.000.000 PRIMEs.

O programa simula 10.000 resultados aleatórios e mostra quanto dinheiro virtual sobrou no final.

2. Estrutura HTML: a interface

A primeira parte do código é responsável por construir a página.

```
None
<h1>Simulador Monte Carlo</h1>
```

Define o título principal.

Depois temos os campos de configuração:

```
None
<input type="number" id="valor" value="100" min="1">
```

Esse campo permite ao usuário informar quantos PRIMEs serão utilizados em cada rodada.

Veja o significado de cada atributo:

- `type="number"`: aceita números.
- `id="valor"`: identifica o campo para o JavaScript.
- `value="100"`: valor inicial.
- `min="1"`: impede valores menores que 1 na interface.

O mesmo princípio é utilizado nos outros campos:

javascript

```
None
valor
rodadas
chance
saldo
```

Esses identificadores permitem que o JavaScript obtenha os valores inseridos pelo usuário.

3. CSS: aparência visual

O CSS controla o design.

Por exemplo:

css

```
None
body {
    font-family: Arial, sans-serif;
    background: #111;
    color: white;
    margin: 0;
    padding: 30px;
}
```

Isso configura:

- Fonte Arial.
- Fundo quase preto.
- Letras brancas.
- Espaçamento da página.

Já este trecho:

css

```
None
.card {
    background: #1e1e1e;
    padding: 25px;
    margin-top: 20px;
    border-radius: 12px;
}
```

Cria os cartões escuros que separam as áreas do simulador.

E este:

css

```
None
@media (max-width: 700px) {
    .config {
        grid-template-columns: 1fr;
    }
}
```

Faz com que os campos fiquem organizados em uma única coluna quando a tela for pequena, como em celulares.

4. JavaScript: o cérebro do simulador

Agora chegamos à parte mais importante.

A função principal é:

javascript

```
None
function simular() {
```

Ela é executada quando o usuário clica no botão:

```
None
<button onclick="simular()">
    Executar Simulação
</button>
```

Ou seja, o clique chama a função `simular()`.

4.1. Capturando os valores

javascript

```
None
const valor = Number(
    document.getElementById("valor").value
);

const rodadas = Number(
    document.getElementById("rodadas").value
);

const chance = Number(
    document.getElementById("chance").value
);

const saldoInicial = Number(
    document.getElementById("saldo").value
);
```

Aqui o programa lê os quatro campos da interface.

O comando `Number()` converte o conteúdo em número para que seja possível realizar operações matemáticas.

4.2. Validando os parâmetros

javascript

```
None
if (
    valor <= 0 ||
    rodadas <= 0 ||
```

```
    saldoInicial <= 0 ||
    chance < 0 ||
    chance > 100
) {
    alert("Digite valores válidos.");
    return;
}
```

Essa condição verifica se os parâmetros são válidos.

Por exemplo:

- Não permite aposta igual a zero.
- Não permite saldo inicial negativo.
- Não permite probabilidade superior a 100%.

O `return` interrompe a execução caso exista algum problema.

4.3. Transformando porcentagem em probabilidade

javascript

```
None
const probabilidadeVitoria = chance / 100;

const probabilidadeDerrota = 1 - probabilidadeVitoria;
```

Essa parte é fundamental.

Se a chance de vitória for 49%, temos:

$$P(V)=49100=0,49$$

$$P(V)=$$

$$100$$

$$49$$

$$=0{,}49$$

A probabilidade de derrota será:

$$\mathrm{P(D)}=1-0{,}49=0{,}51$$

$$P(D)=1-0{,}49=0{,}51$$

Portanto:

- Vitória: 49%.
- Derrota: 51%.

A soma das probabilidades sempre será 100%.

5. O cálculo matemático: valor esperado

Essa é uma das partes mais importantes do simulador.

O código utiliza:

javascript

```
None
const valorEsperado =
    (probabilidadeVitoria * valor) +
    (probabilidadeDerrota * -valor);
```

O que ele está calculando?

O valor esperado representa o resultado médio teórico de uma rodada quando o experimento é repetido muitas vezes.

A fórmula é:

$$\mathrm{E[X]=P(V)\cdot G+P(D)\cdot P}$$

$$E[X]=P(V)\cdot G+P(D)\cdot P$$

Onde:

- $\mathrm{P(V)}$

- $P(V)$: probabilidade de vitória.
- G
- G: ganho na vitória.
- P(D)
- $P(D)$: probabilidade de derrota.
- P
- P: resultado financeiro da derrota, neste caso negativo.

Vamos utilizar os números do exemplo:

$$\mathrm{E[X]} = (0{,}49 \times 100) + (0{,}51 \times -100)$$

$$E[X] = (0{,}49 \times 100) + (0{,}51 \times -100)$$

$$\mathrm{E[X]} = 49 - 51$$

$$E[X] = 49 - 51$$

Resultado esperado por rodada

-2 PRIMEs

Perda média teórica por rodada.

Isso significa que, matematicamente, cada rodada representa uma perda média esperada de 2 PRIMEs.

5.1. Expectativa total

O código também calcula:

javascript

```
None
const expectativaTotal = valorEsperado * rodadas;
```

Com 10.000 rodadas:

$$\mathrm{E[T]} = -2 \times 10.000$$

$$E[T] = -2 \times 10.000$$

$$\mathrm{E[T]} = -20.000$$

$$E[T] = -20.000$$

Isso significa que a perda média teórica acumulada é de 20.000 PRIMEs.

Importante: não significa que toda simulação terminará exatamente com essa perda. O resultado real depende dos sorteios aleatórios.

6. O algoritmo Monte Carlo

Agora chegamos ao mecanismo que efetivamente simula as rodadas.

Primeiro, o programa inicializa algumas variáveis:

javascript

```
None
let saldo = saldoInicial;

let vitorias = 0;
let derrotas = 0;

let historico = [saldo];
```

Cada variável tem uma função:

- `saldo`: saldo atual.
- `vitorias`: quantidade de vitórias.
- `derrotas`: quantidade de derrotas.
- `historico`: armazena a evolução do saldo.

Depois começa o laço:

javascript

```
None
for (let i = 1; i <= rodadas; i++) {
```

Esse comando repete o código enquanto o número de rodadas não for atingido.

Dentro dele está a parte mais importante:

javascript

```
None
const sorteio = Math.random();

if (sorteio < probabilidadeVitoria) {
    saldo += valor;
    vitorias++;
} else {
    saldo -= valor;
    derrotas++;
}
```

Como funciona o Math.random()?

O `Math.random()` gera um número aleatório entre 0 e 1.

Por exemplo:

Sorteio	Resultado
0,12	Vitória
0,37	Vitória
0,48	Vitória
0,53	Derrota
0,91	Derrota

Com probabilidade de vitória de 49%, qualquer sorteio inferior a 0,49 representa uma vitória.

Qualquer sorteio igual ou superior a 0,49 representa uma derrota.

Quando vence:

javascript

```
None
saldo += valor;
vitorias++;
```

O saldo aumenta e o contador de vitórias é incrementado.

Quando perde:

javascript

```
None
saldo -= valor;
derrotas++;
```

O saldo diminui e o contador de derrotas aumenta.

Finalmente:

javascript

```
None
historico.push(saldo);
```

O saldo atualizado é armazenado para construir o gráfico posteriormente.

7. Terminal da simulação

O código utiliza:

javascript

```
None
terminal.textContent +=
    "Rodada #" + i +
    " | Saldo virtual: " + saldo;
```

Esse trecho adiciona informações ao terminal.

Mas existe um detalhe interessante: o programa não mostra todas as rodadas.

Ele utiliza:

javascript

```
None
if (
```

```
    i === Math.floor(rodadas * 0.1) ||
    i === Math.floor(rodadas * 0.5) ||
    i === rodadas
)
```

Isso significa que apresenta informações em três momentos:

- 10% da simulação.
- 50% da simulação.
- 100% da simulação.

Assim, o terminal não fica excessivamente cheio.

8. Calculando o resultado final

Depois que todas as rodadas terminam, o programa executa:

javascript

```
None
const resultado = saldo - saldoInicial;

const percentual =
    (resultado / saldoInicial) * 100;
```

O primeiro cálculo identifica quanto o saldo aumentou ou diminuiu.

O segundo calcula a variação percentual em relação ao saldo inicial.

Exemplo:

- Saldo inicial: 1.000.000.
- Saldo final: 980.000.

Resultado:

$$980.000 - 1.000.000 = -20.000$$

$$980.000 - 1.000.000 = -20.000$$

Percentual:

$$-20.0001.000.000 \times 100 = -2\%$$

$$1.000.000$$

$$-20.000$$

$$\times 100 = -2\%$$

Portanto, o saldo diminuiu 2%.

9. O gráfico de evolução do saldo



A função responsável pelo gráfico é:

javascript

```
function desenharGrafico(dados, saldoInicial) {
```

Ela recebe duas informações:

- `dados`: histórico completo dos saldos.
- `saldoInicial`: valor de referência.

Primeiro, recupera o elemento Canvas:

javascript

```
None
const canvas =
    document.getElementById("grafico");

const ctx = canvas.getContext("2d");
```

O Canvas permite desenhar gráficos diretamente no navegador.

Depois, encontra os valores extremos:

javascript

```
None
let minimo = Math.min(...dados);

let maximo = Math.max(...dados);
```

Isso permite ajustar a escala vertical do gráfico aos resultados da simulação.

Em seguida, desenha três elementos:

1. Os eixos horizontal e vertical.
2. Uma linha pontilhada indicando o saldo inicial.
3. Uma linha azul mostrando a evolução do saldo.

A linha azul é construída utilizando as coordenadas de cada posição do histórico.

Assim, é possível visualizar as oscilações durante as rodadas.

10. O que significa Monte Carlo?

O método

Monte Carlo é uma técnica matemática e computacional que utiliza números aleatórios para estudar fenômenos incertos.

Em vez de tentar prever um único resultado, ele executa muitos experimentos e analisa os resultados obtidos.

É utilizado em áreas como:

- Finanças e análise de riscos.
- Engenharia.
- Física.

- Pesquisa científica.
- Estatística.
- Modelagem de sistemas complexos.

No seu simulador, cada rodada representa um experimento aleatório.

10.000 rodadas representam 10.000 experimentos.

Quanto maior a quantidade de simulações, melhor podemos observar o comportamento estatístico do modelo, embora uma única execução ainda possa apresentar variações consideráveis.

11. Uma observação matemática muito importante

Existe algo interessante no seu simulador.

Ele considera que o ganho e a perda são iguais, mas a probabilidade de vitória é de apenas 49%.

Isso produz uma expectativa negativa.

Veja:

Chance de vitória	Resultado esperado por rodada (aposta de 100)
40%	-20 PRIMEs
45%	-10 PRIMEs
49%	-2 PRIMEs
50%	0 PRIMEs
55%	+10 PRIMEs
60%	+20 PRIMEs

Gráfico

Valor esperado por rodada, considerando ganho e perda de 100 PRIMEs.

	Valor esperado
40%	-20 PRIMEs
45%	-10 PRIMEs
49%	-2 PRIMEs
50%	0 PRIMEs
55%	10 PRIMEs
60%	20 PRIMEs

O ponto de equilíbrio matemático é 50%.

Com 49% de chance de vitória, existe uma desvantagem matemática de 2% sobre o valor apostado em cada rodada.

Isso é chamado de expectativa negativa.

12. Limitações do código atual

Seu simulador funciona como demonstração matemática, mas existem alguns pontos que merecem atenção:

1. Não existe limite de saldo: o programa permite que o saldo fique negativo.
2. Não existe parada por insolvência: mesmo com saldo negativo, as rodadas continuam.
3. O histórico pode consumir muita memória: um milhão de rodadas gera um milhão de pontos armazenados.
4. Não calcula desvio padrão: portanto, não mostra diretamente a dispersão dos resultados.
5. Não calcula probabilidade de ruína: não estima a frequência com que o saldo chega a zero em várias simulações independentes.
6. Não utiliza semente aleatória: cada execução pode produzir resultados diferentes e não é facilmente reproduzível.

1. A casa de apostas tem uma vantagem matemática

Imagine um jogo simples:

- Se o apostador ganhar, recebe 100 PRIMEs de lucro.
- Se perder, perde 100 PRIMEs.
- A probabilidade real de vitória é 49%.
- A probabilidade real de derrota é 51%.

A fórmula do valor esperado é:

$$\mathrm{E[X]}=(0{,}49\times 100)+(0{,}51\times -100)$$

$$E[X]=(0{,}49\times 100)+(0{,}51\times -100)$$

$$\mathrm{E[X]}=49-51$$

$$E[X]=49-51$$

$$\mathrm{E[X]}=-2$$

$$E[X]=-2$$

Isso significa que o apostador perde, em média, 2 PRIMEs por rodada.

A casa tem uma expectativa de ganho de 2 PRIMEs por rodada, considerando apostas de 100 PRIMEs e esse modelo simplificado.

A matemática de cada 100 PRIMEs apostados

Retorno esperado do apostador

98 PRIMEs

Vantagem matemática da casa

2%

Modelo ilustrativo de apostas com pagamentos iguais e probabilidades de 49% contra 51%.

2. O segredo está nas odds

Agora vamos imaginar uma aposta esportiva.

Suponha que dois resultados tenham exatamente 50% de probabilidade cada.

Se a casa fosse oferecer odds perfeitamente justas, seriam:

- Resultado A: odd 2,00.
- Resultado B: odd 2,00.

Mas a casa pode oferecer:

- Resultado A: odd 1,90.
- Resultado B: odd 1,90.

O que acontece?

Imagine que você aposte 100 reais.

Se ganhar:

$$100\times1{,}90=190$$

$$100\times1{,}90=190$$

Você recebe R$ 190, incluindo os R$ 100 apostados.

Seu lucro é de R$ 90.

Se perder, perde R$ 100.

Agora calculamos:

$$\mathrm{E[X]}=(0{,}5\times90)+(0{,}5\times-100)$$

$$E[X]=(0{,}5\times90)+(0{,}5\times-100)$$

$$\mathrm{E[X]}=45-50$$

$$E[X]=45-50$$

$$\mathrm{E[X]}=-5$$

$$E[X]=-5$$

Portanto, o apostador perde em média R$ 5 para cada R$ 100 apostados nesse modelo.

A casa tem uma vantagem esperada de 5%.

3. Por que o tempo favorece a casa?

Aqui está o ponto mais importante.

Uma pessoa pode ganhar uma aposta, dez apostas ou até cem apostas consecutivas.

Mas, quando existe uma vantagem matemática constante para a casa, o volume de apostas faz essa vantagem aparecer estatisticamente.

Vamos utilizar o exemplo anterior:

- Aposta: R$ 100.
- Probabilidade de vitória: 50%.
- Lucro na vitória: R$ 90.
- Prejuízo na derrota: R$ 100.

Gráfico

Expectativa acumulada do apostador, considerando 5% de vantagem matemática da casa sobre o valor apostado.

	Resultado esperado
1	R$ -5
100	R$ -500
1.000	R$ -5 mil

10.000	R$ -50 mil

Observe:

Apostas	Perda esperada
1	R$ 5
100	R$ 500
1.000	R$ 5.000
10.000	R$ 50.000

Isso não significa que cada pessoa necessariamente perderá esses valores. São expectativas matemáticas, não resultados garantidos.

Mas demonstra por que a vantagem da casa é tão importante.

4. O efeito psicológico: o apostador não enxerga a matemática

Existe outro fator importante.

O ser humano costuma prestar mais atenção nas vitórias do que nas perdas.

Imagine este cenário:

1. O apostador perde R$ 100.
2. Depois ganha R$ 190.
3. Fica animado porque ganhou R$ 90.
4. Continua apostando.
5. Perde várias apostas posteriormente.

O problema é que uma vitória isolada não elimina a vantagem matemática da casa.

Outro fenômeno é a tentativa de recuperar perdas.

Por exemplo:

- Perde R$ 100.

- Aposta R$ 200 para recuperar.
- Perde R$ 200.
- Aposta R$ 400 para recuperar.

Isso pode acelerar muito o esgotamento do saldo.

5. O que acontece com o saldo ao longo do tempo?

Vamos voltar ao seu simulador.

Ele utiliza:

javascript

```
None
const sorteio = Math.random();

if (sorteio < probabilidadeVitoria) {
    saldo += valor;
    vitorias++;
} else {
    saldo -= valor;
    derrotas++;
}
```

Esse pequeno trecho representa o mecanismo matemático de uma vantagem probabilística.

A casa não precisa determinar qual será o próximo resultado.

Ela simplesmente opera um sistema em que a expectativa matemática favorece seu lado.

E quanto mais apostas independentes forem realizadas, maior será a tendência de o resultado acumulado refletir essa expectativa.

6. Existe uma diferença entre ganhar e ter vantagem matemática

Essa distinção é fundamental:

Situação	Significado
Ganhar uma aposta	Resultado positivo isolado
Ganhar 10 apostas	Sequência positiva
Ter vantagem matemática	Expectativa positiva por aposta
Ter desvantagem matemática	Expectativa negativa por aposta

Uma pessoa pode ganhar dinheiro durante uma sessão mesmo estando em desvantagem matemática.

Da mesma forma, uma casa pode ter prejuízo em determinado dia, apesar de possuir vantagem matemática sobre o conjunto das apostas.

A vantagem estatística não garante o resultado de cada evento.

7. A grande lição do seu simulador

Seu código demonstra um princípio muito importante:

Não é necessário perder em todas as rodadas para perder dinheiro no longo prazo.

Basta que o valor esperado seja negativo.

E aqui está uma diferença essencial:

- O apostador normalmente observa cada vitória e derrota individualmente.
- A casa administra o conjunto de milhares ou milhões de apostas.

A casa trabalha com uma vantagem estatística agregada, enquanto o apostador individual está exposto à variabilidade dos resultados.

A fórmula que explica tudo

$$\mathrm{E[X]=P(V)\cdot G-P(D)\cdot L}$$

$$E[X]=P(V)\cdot G-P(D)\cdot L$$

Onde:

- $\mathrm{P(V)}$
- $P(V)$ = probabilidade de vitória.
- G
- G = lucro quando ganha.
- $\mathrm{P(D)}$
- $P(D)$ = probabilidade de derrota.
- L
- L = prejuízo quando perde.

Se

$\mathrm{E[X]}$

$E[X]$ for negativo, o apostador tem desvantagem matemática.

Se

$\mathrm{E[X]}$

$E[X]$ for positivo, o apostador tem vantagem matemática teórica.

Se

$\mathrm{E[X]=0}$

$E[X]=0$, o jogo é matematicamente equilibrado antes de outros custos.

Uma última observação: não é correto dizer que absolutamente toda pessoa sempre perde para a casa. Existem apostadores que terminam no lucro, inclusive no longo prazo. Mas, quando as odds incorporam uma margem para a casa, o conjunto dos apostadores enfrenta uma expectativa matemática desfavorável.